

Herald



Herald — Telegram Integration (Operator Guide)

Field	Value
Revision	1
Created	2026-05-28
Last modified	2026-05-28
Status	active
Status summary	Operator-facing deep-dive for Herald's Telegram integration — Herald's most-used messenger channel, live since Wave 6 (HRD-011 outbound substrate + HRD-100 inbound runtime). Documents the two-way model (inbound <code>pherald listen</code> → <code>getUpdates</code> long-poll → Claude Code dispatch → outbound reply via <code>bot.Send</code> / <code>bot.SendReply</code>), the BotFather walkthrough (bot creation, privacy-mode toggling), every required env var with concrete acquisition steps, the end-to-end verification recipes (<code>getMe</code>, <code>getChat</code>, <code>hermetic go test</code>, <code>pherald listen smoke</code>), Bot API hard limits (4096-char text, 50MiB documents, 10MiB photos free / 50MiB premium), the per-channel content-addressed inbox (<code>~/ .herald/inbox/tgram/<sha256>.<ext></code>), chat-type taxonomy (<code>private</code> / <code>group</code> / <code>supergroup</code> / <code>channel</code> — what <code>chat_id</code> looks like for each), reply threading + attachment fan-out (the M3 rationale for serialised replies over multi-attachment albums), the self-filter that prevents echo loops (<code>bot.Me.Username</code> cached at Subscribe boot — fail-loud if empty), a 10-entry troubleshooting cookbook (401/403/409/413/429 + privacy-mode + webhook conflicts + zero-byte downloads), the operator pre-deploy audit checklist, and the full cross-reference index to spec V3, HRDs, source files, and sibling guides.
Issues	none
Issues summary	—
Fixed	(n/a — new guide)
Continuation	bump when MTPProto user-client lands (Wave 7+ — current bot-to-bot wall workaround captured in <code>feedback memory telegram_bot_to_bot_wall</code>); bump when local Bot API server support lands (lifts the 50MiB document cap); bump when forum-topic threading is rounded out beyond V1 stub; bump when interactive call routing (<code>Capabilities.InteractiveCall</code>) lands; bump when webhook ingress (Step 7 in <code>messenger s/TELEGRAM.md</code>) is implemented.

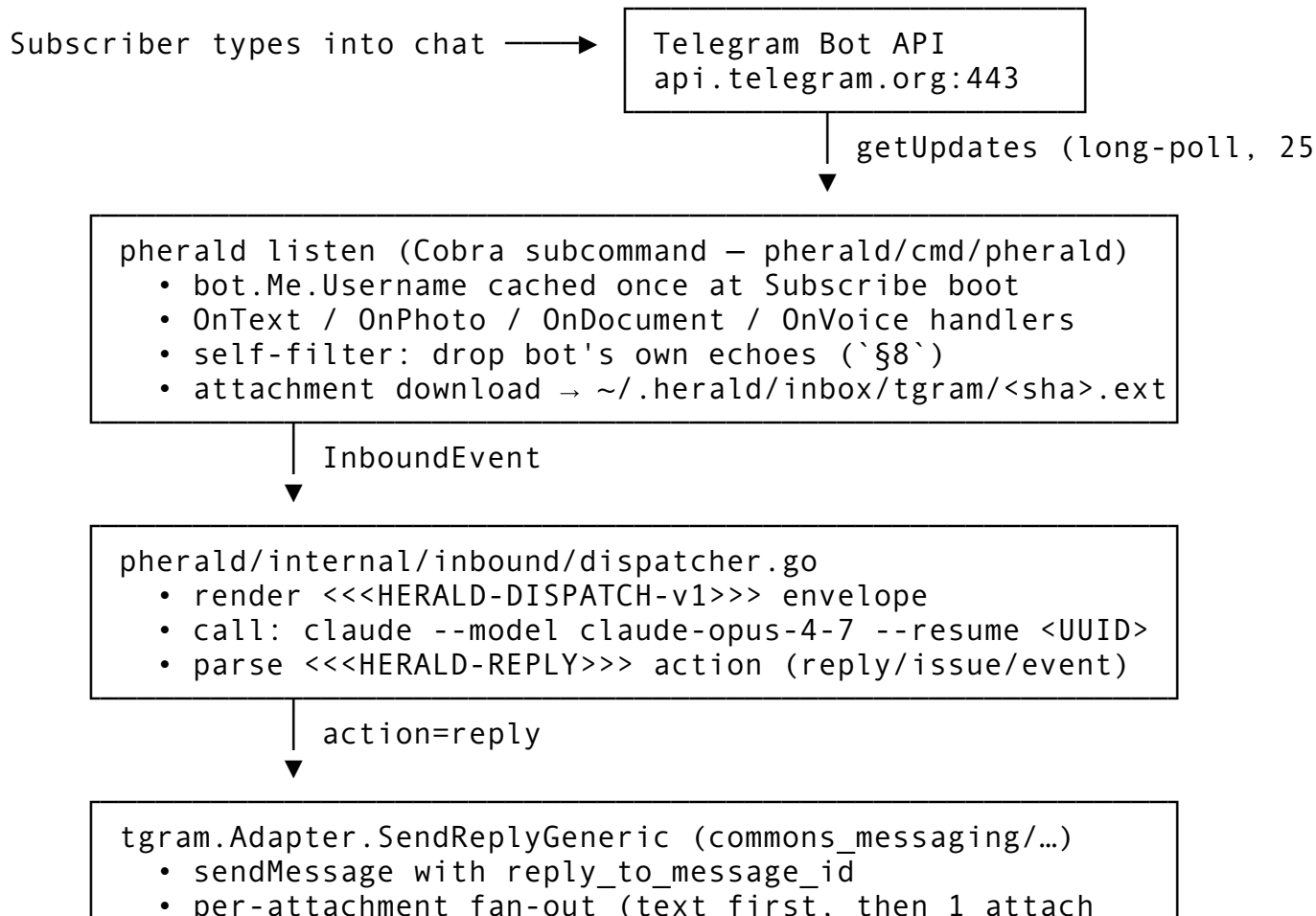
Table of contents

- [§1. What is Herald's Telegram integration?](#)
 - [§2. BotFather walkthrough \(creating a bot\)](#)
 - [§3. Required env vars + how to obtain each](#)
 - [§4. Verifying the integration end-to-end](#)
 - [§5. Attachment size + type limits](#)
 - [§6. Chat types — private vs group vs supergroup vs channel](#)
 - [§7. Reply threading + `reply\_to\_message\_id`](#)
 - [§8. Self-filter — preventing echo loops](#)
 - [§9. Troubleshooting cookbook](#)
 - [§10. Pre-deploy operator audit](#)
 - [§11. References](#)
-

§1. What is Herald's Telegram integration?

Telegram is Herald's reference messenger — the first channel that went live (Wave 6, 2026-05-21) and the channel against which every cross-cutting invariant in the messaging plane was first proven. It is bidirectional: Herald can both deliver outbound notifications to a chat AND react to inbound messages a subscriber posts in that same chat.

§1.1 Two halves of one channel



The outbound half (`Send` / `SendReply` / `SendReplyGeneric`) is available to ANY Herald flavor binary — `pherald` `serve`, `sherald`, `cherald`, `bherald`, `rherald`, `iherald`, `scherald`, `qaherald` — through the `commons.Channel` outbound contract (spec §11.0). The inbound half is exclusive to `pherald listen` (and, by extension, `qaherald` which embeds the same inbound runtime for QA round-trips).

§1.2 Status

- **Outbound** — LIVE since Wave 6 (HRD-011). Hermetic tests + Wave 6.5 paired §1.1 mutation gate pin the wire-byte shape. Live evidence: `docs/qa/HRD-011-LIVE-*/` (operator-supplied).
- **Inbound** — LIVE since Wave 6 (HRD-100). `pherald listen` Cobra subcommand drives the closed-loop runtime; T10a --qa-out-dir JSONL journaling captures every dispatch. Live evidence: `docs/qa/HRD-100-LIVE-*/` (operator-supplied; T10b currently open until live transcripts land).
- **Multi-channel fan-in** — LIVE since Wave 7 (HRD-114). Telegram now coexists with Slack under one shared inbound dispatcher; see `MESSENGER_CHANNELS.md`.

§1.3 Where the code lives

- `commons_messaging/channels/tgram/tgram.go` — Adapter struct, `New` / `NewWithCreds` constructors, `init()` registry binding (Wave 7 T2), `BotSelfIdentity` / `SendReplyGeneric` / `DownloadAttachment` channel-interface methods.
- `commons_messaging/channels/tgram/send.go` — outbound `Send` + native `SendReply(ctx, chatID, body, replyToID, attachments)`.
- `commons_messaging/channels/tgram/subscribe.go` — inbound `Subscribe(ctx, h)` long-poll loop, `stampAndIsSelfEcho` filter, `OnText` / `OnPhoto` / `OnDocument` / `OnVoice` handlers.
- `commons_messaging/channels/tgram/attachments.go` — `DownloadAttachment(ctx, bot, fileID, mime)` → content-addressed write into `~/.herald/inbox/tgram/<sha>.<ext>`.
- `commons_messaging/channels/tgram/healthcheck.go` — `HealthCheck` (calls `getMe`).
- `commons_messaging/channels/tgram/persist.go` — outbound-delivery-evidence persistence into the `outbound_delivery_evidence` Postgres table (RLS-aware via `SendForTenant`).
- `pherald/cmd/pherald/listen.go` — the `pherald listen` Cobra subcommand that wires the inbound runtime.
- `pherald/internal/inbound/dispatcher.go` — Claude Code dispatch + `<<<HERALD-REPLY>>>` action router.

§2. BotFather walkthrough (creating a bot)

A Telegram **bot** is a user-like entity owned by your real Telegram account. You create it by chatting with `@BotFather`, Telegram's official bot-management bot.

§2.1 Create the bot

1. Open Telegram (mobile, desktop, or web).
2. Tap the search icon and search BotFather.
3. Open the chat with the verified-blue-checkmark account @BotFather — DO NOT pick any impostor BotFatherSomething accounts.
4. Send `/start` (one-time intro).
5. Send `/newbot`.
6. BotFather replies asking for the bot's **display name**. Pick a human-readable string (any UTF-8). Examples: Herald Ops, My Project CI, Acme Notifications.
7. BotFather then asks for the bot's **username**. Must be globally unique on Telegram and MUST end with `bot` or `_bot`. Examples: `herald_ops_bot`, `myproject_ci_bot`, `acme_notify_bot`.
8. BotFather replies with a token of the canonical form:

```
1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t
```

The 1234567890 prefix is the bot's numeric user id; the AAH... portion is the HMAC the Bot API uses to authenticate API calls. Treat the entire string as a password.

9. **Copy the token into your password manager immediately** — BotFather will display it once and only show subsequent reads on demand via `/mybots` → `<select bot>` → API Token.

§2.2 Privacy mode

By default, every new Telegram bot is created with **privacy mode ENABLED**. With privacy on, a bot in a group can see ONLY (a) commands directed at it (e.g. `/foo`), (b) messages that mention it by `@username`, and (c) replies to its own messages. Regular group chatter is invisible to the bot's `getUpdates`.

For Herald, the standard Wave 6 use-case is "DM with the bot, the bot relays the conversation to Claude Code" — which works fine with privacy mode ON (private DMs are always fully visible to the bot regardless of the privacy setting). But if you want the bot to live in a group and observe ALL traffic, you MUST disable privacy:

1. DM @BotFather.
2. Send `/setprivacy`.
3. Select your bot from the inline keyboard.
4. Choose `Disable`.

Privacy-mode-on is the single most common cause of "the bot doesn't see my group messages" support tickets. Always start with privacy-mode-off if the bot lives in a group.

§2.3 Add the bot to a chat

For a 1:1 DM (the simplest path — preferred for `herald listen smoke`):

1. In Telegram, search for the bot's `@username`.
2. Tap **Start** to open a DM. The `/start` button (auto-injected by Telegram's client for first-contact with a bot) registers YOU as a chat the bot knows about.
3. Send any plain-text message (e.g. `hello`).

For a group / supergroup:

1. Open the group, tap the title to enter group settings → **Add Members**.
2. Search for your bot's @username, tap to add.
3. (Optional, recommended for non-admin bots in supergroups) Promote the bot to **Admin** so it can send messages (in some channel configurations bots must be admins to post — see §6 for channel chat type specifics).
4. Send a test message in the group.

For a channel (one-way broadcast — Herald sends, no inbound):

1. Open the channel, channel settings → **Administrators** → **Add Administrator**.
2. Add the bot. The channel administrator scopes (Post Messages, Edit Messages of Others, Delete Messages of Others) are sufficient — Herald does not need anything more.
3. (Channels do not generate getUpdates inbound events that flow through Herald's current path — inbound is private + group + supergroup only.)

§2.4 Optional BotFather knobs

Other commands you may want to set once via BotFather:

Command	Purpose
/setname	Change the display name.
/setdescription	Long-form bot description shown in the bot's profile page.
/setabouttext	One-line tagline shown next to the bot's name in the "share contact" UI.
/setuserpic	Upload an avatar (recommended — set the Herald logo).
/setcommands	Register /foo-style commands so the Telegram client renders an inline / menu. Herald does not currently require this.
/deletebot	Permanently delete the bot. Irreversible.
/revoke	Invalidate the current token + issue a fresh one. Use this immediately if the token leaks.

§3. Required env vars + how to obtain each

Herald's Telegram channel reads exactly two env vars at boot:

Variable	Required for	Format	Example
HERALD_TGRAM_BOT_TOKEN	outbound + inbound	<bot-id>:<api-token> from BotFather	1234567890:AAH1lab2c3d4e5f6g7h8i9j
HERALD_TGRAM_CHAT_ID	outbound default destination	numeric (private = positive; group = negative;	-1001234567890

Variable	Required for	Format	Example
		supergroup = - 100... prefix; channel = - 100... prefix)	

Optional / reserved (NOT required for Wave 6/7 functionality):

Variable	Required for	Notes
HERALD_TGRAM_LIVE_INBOUND	integration-test gating	Set to 1 to enable live inbound integration tests in <code>commons_messaging/channels/tgram/subscribe_integration_test.go</code> .
HERALD_TGRAM_WEBHOOK_SECRET	future webhook ingress	Reserved for a future HRD; webhook ingress is currently NOT implemented. Long-poll via <code>getUpdates</code> is the only inbound transport.
HERALD_INBOX_DIR	attachment storage	Overrides the per-channel inbox parent dir (default: <code>~/herald/inbox/</code>). Per-channel subdir name is fixed to <code>tgram/</code> .

§3.1 Obtaining HERALD_TGRAM_BOT_TOKEN

The bot token is the canonical string BotFather emits at `/newbot` (see §2.1 step 8). It is the SAME string returned by `/mybots → <bot> → API Token` on demand. If lost, rotate via `/revoke` — there is no recovery path.

§3.2 Obtaining HERALD_TGRAM_CHAT_ID

For a **1:1 DM** (positive integer):

1. Open the DM with your bot.
2. Send any plain-text message (e.g. `id-probe`).
3. From a shell with `curl`:

```
curl -s "https://api.telegram.org/bot$
      {HERALD_TGRAM_BOT_TOKEN}/getUpdates" | jq
      '.result[].message.chat'
```

4. Read `.id` — that integer (positive for DMs) is your `HERALD_TGRAM_CHAT_ID`.

For a **group** (negative integer):

1. Add the bot to the group (§2.3).
2. Send any message in the group (e.g. `@your_bot_username id-probe` if privacy mode is ON — otherwise any plain text works).
3. Same `getUpdates` curl as above; the `chat.id` is a negative integer.

For a **supergroup** (negative integer starting with -100):

1. Same as group (Telegram auto-converts groups to supergroups when admin features are enabled).
2. The chat id begins with the literal prefix -100 followed by the numeric body, e.g. -1001234567890.

For a **channel** (negative integer starting with -100):

1. Add the bot as channel admin (§2.3).
2. Post a message to the channel.
3. Same `getUpdates` curl; if the bot has post permissions it will receive the `channel_post` update with `.channel_post.chat.id`.

Helper utility bots (alternative to manual curl): add `@userinfobot` or `@RawDataBot` to your chat — they reply with the chat id as plain text. These are third-party bots; vet them before sharing sensitive chat content.

Herald-provided diagnostic (if installed): `bash scripts/tgram_diagnose.sh` is the canonical sanity script in this repo. It calls `getUpdates`, parses the response, and prints chat ids in human-friendly form. Use it during onboarding.

§3.3 Copy-pastable `.env` snippet

```
# Herald — Telegram (HRD-011 outbound + HRD-100 inbound)
export HERALD_TGRAM_BOT_TOKEN='1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3o'
export HERALD_TGRAM_CHAT_ID='-1001234567890'

# Optional (defaults shown):
# export HERALD_INBOX_DIR="$HOME/.herald/inbox"
# export HERALD_TGRAM_LIVE_INBOUND='1'           # only to enable live
integration tests
```

Per `OPERATOR_CREDENTIALS.md` §"Resolution order (12-factor)", exported shell vars take precedence over `.env`. Put production tokens in your shell rc; put dev/test tokens in `.env` (the project-local `.env` is gitignored).

§4. Verifying the integration end-to-end

There are four progressive verification steps. Each is cheap; each catches a different class of misconfiguration. Run them in order before promoting a deploy.

§4.1 Pre-flight: probe the Bot API directly

The cheapest sanity check is a raw `getMe` curl — it proves the token is valid + the API endpoint is reachable + your network can talk to Telegram.

```
curl -s "https://api.telegram.org/bot${HERALD_TGRAM_BOT_TOKEN}/getMe" | jq
```

A healthy response looks like:

```
{
  "ok": true,
  "result": {
    "id": 1234567890,
    "is_bot": true,
    "first_name": "Herald Ops",
    "username": "herald_ops_bot",
    "can_join_groups": true,
    "can_read_all_group_messages": false,
    "supports_inline_queries": false
  }
}
```

Look for `ok: true` and `result.username` populated. If `ok: false`, the token is wrong or revoked (see §9.1). The `can_read_all_group_messages: false` field reports the privacy-mode setting — `true` means you successfully disabled privacy via `/setprivacy`.

§4.2 Verify the bot can see the target chat

```
curl -s "https://api.telegram.org/bot${HERALD_TGRAM_BOT_TOKEN}/
  getChat?chat_id=${HERALD_TGRAM_CHAT_ID}" | jq
```

Healthy response:

```
{
  "ok": true,
  "result": {
    "id": -1001234567890,
    "type": "supergroup",
    "title": "Herald Ops",
    "permissions": { "...": "..." }
  }
}
```

If you see `{"ok": false, "error_code": 400, "description": "Bad Request: chat not found"}`, either the chat id is wrong-format (see §6 for the per-type shape) or the bot has not been added to the chat (see §2.3).

§4.3 Verify the pherald binary is reachable

```
go build -o /tmp/pherald ./pherald/cmd/pherald
/tmp/pherald version
```

Expected output:

```
pherald X.Y.Z (commit <sha>, built <date>)
```

If the binary builds but `version` panics, the workspace is broken (see `CLAUDE.md` § build + test). If the binary exits non-zero, file a bug — `version` is the simplest possible subcommand and must always work.

§4.4 Hermetic Send + persistence test

With `HERALD_TGRAM_BOT_TOKEN` + `HERALD_TGRAM_CHAT_ID` set, and a Postgres container running (see `docs/guides/OPERATOR_CREDENTIALS.md` § "Postgres" for the quickstart compose flow):

```
# Bring up Postgres if needed (one of these):
#   ./quickstart/up.sh                # full stack
#   bash containers/scripts/up.sh pg  # just Postgres

# Run the integration test – actually crosses the wire to
#   Telegram:
go test -tags=integration -run TestSend_PersistsDeliveryEvidence \
    ./commons_messaging/channels/tgram/...
```

A PASS confirms: (a) the bot token is accepted by `api.telegram.org`; (b) the chat id resolves; (c) `sendMessage` returns a chat-side `message_id`; (d) the `outbound_delivery_evidence` row is persisted in Postgres with the correct evidence ceiling (`DeliveryRouted`); (e) the LIVE Bot API end-to-end works.

§4.5 Inbound smoke (pherald listen)

The end-to-end smoke: run `pherald listen` in the foreground, type a message in the chat, observe Claude Code dispatch + reply.

```
# Optional: capture the dispatch journal for §107.x evidence
mkdir -p docs/qa/HRD-100-smoke-$(date -u +%Y%m%dT%H%M%SZ)/

/tmp/pherald listen \
    --qa-out-dir docs/qa/HRD-100-smoke-$(date -u +
    %Y%m%dT%H%M%SZ) \
    2>&1 | tee /tmp/pherald-listen.log
```

Boot-time lines to look for in `/tmp/pherald-listen.log`:

```
pherald listen: starting Telegram getUpdates long-poll loop
pherald listen: bot.Me.Username=herald_ops_bot (self-filter active)
```

If `bot.Me.Username=` is empty, Subscribe will refuse to boot (see §8). Send a test message in the Telegram chat — within ~25s you should see a `<<<HERALD-DISPATCH-v1>>>` envelope being rendered to Claude Code in the log, followed by the bot's reply landing in your Telegram chat.

Tear down with `Ctrl-C` (graceful shutdown). Confirm the `--qa-out-dir` directory contains a `journal.jsonl` with one line per dispatch.

§5. Attachment size + type limits

The Bot API enforces strict hard caps that Herald inherits. Knowing them in advance avoids the silently-trimmed-or-rejected surprise at first prod use.

§5.1 Hard caps (uploads via the bot)

Type	Bot API endpoint	Hard limit	Notes
Text	sendMessage	4096 chars	UTF-16 code unit count, not byte count. MarkdownV2 escape characters count toward the limit. Herald does NOT auto-split — over-limit messages return <code>400 Bad Request: message is too long</code> .
Photo	sendPhoto	10 MB (free tier)	Photo uploads compress server-side; effective ceiling is ~10MB regardless of user account tier when the bot uploads.
Document	sendDocument	50 MB	The hardest cap. Holds even for premium users when uploaded BY THE BOT through the public Bot API. The workaround is the local Bot API server (<code>telegram-bot-api</code> self-hosted) — see §5.4.
Voice	sendVoice	50 MB	Same as document; voice messages are OGG/Opus encoded.
Video	sendVideo	50 MB	Same as document.
Audio	sendAudio	50 MB	Same as document.

§5.2 User-side download caps (inbound attachments)

When a subscriber sends a file to the bot, Herald reads it via `getFile` + the file-download endpoint. The bot can **DOWNLOAD** files up to **20 MB** through the standard Bot API. Larger files (which premium users can **SEND** in Telegram) trigger `Bad Request: file is too big` on `getFile`. This is a Bot-API-side limit; the workaround is again the local Bot API server.

§5.3 The content-addressed inbox

Every successfully-downloaded inbound attachment lands at:

```
${HERALD_INBOX_DIR:-$HOME/.herald/inbox}/tgram/<sha256-hex>.<ext>
```

The sha256 IS the filename stem; the extension is derived from the MIME via `channels.MimeToExt` (intentionally narrow — `.jpg`, `.png`, `.pdf`, `.ogg`, `.mp3`, `.mp4`, `.bin` fallback for unknown MIMEs). Idempotence: a duplicate download of identical bytes is a no-op (the existing file IS the proof of presence), so the long-poll never burns Bot API quota re-downloading the same file.

The streaming write helper is `channels.WriteContentAddressed("tgram", mime, body)` in `commons_messaging/channels/inbox.go`. It uses `io.MultiWriter(tempFile, sha256Hasher)` so the body flows to disk while the sha256 is computed inline; the final `os.Rename` is atomic, so partial writes are never visible at the content-addressed path. The temp file naming is `dl-*.*.part`; the next download starts fresh from a missing-temp state. This invariant is pinned by `TestDownloadAttachmentContentAddressed` in `commons_messaging/channels/tgram/attachments_test.go`.

§5.4 Lifting the 50 MB ceiling — local Bot API server

The 50 MB document upload cap and 20 MB download cap are properties of `api.telegram.org` (the public Bot API server). Telegram open-sourced the Bot API server itself; running a local instance against your own MTProto credentials raises the caps:

- **Document upload:** up to 2000 MB.
- **Document download:** full file size (Telegram's storage layer has no Bot API-level cap when the API server is local).

The local Bot API server is currently OUT OF SCOPE for Herald (HRD-NNN reserved). When it lands, Herald will route Bot API calls through a configurable `HERALD_TGRAM_BOT_API_URL` env var (default `https://api.telegram.org`).

§5.5 Where these limits surface in HRDs

HRD	Symptom
-----	---------

HRD-011	Outbound substrate — first to encounter the 50 MB cap during attachment fan-out tests.
HRD-100	Inbound runtime — first to encounter the 20 MB download cap on real subscriber-sent files.
HRD-134	Tgram-completeness — the structured rationale for NOT auto-splitting 4096+ char messages (the user must explicitly opt in to a splitter; silent splits are a §107 surprise).

§6. Chat types — private vs group vs supergroup vs channel

Telegram has four chat types Herald cares about. Each has a different `chat_id` shape and a different permission model.

§6.1 Chat-type taxonomy

Type	<code>chat_id</code> shape	Example	Notes
private	positive integer, equal to the user's <code>user_id</code>	123456789	1:1 DM between the bot and exactly one Telegram user. Privacy mode is irrelevant — DMs are always fully visible.
group	negative integer (no specific prefix)	-123456789	"Basic group" — legacy format, max 200 members, no admin features. Privacy mode applies.
supergroup	negative integer starting with -100	-1001234567890	Modern group format, up to 200k members, full admin features. Privacy mode applies. Groups auto-convert to supergroups when admins are added.
channel	negative integer starting with -100	-1009876543210	One-way broadcast. The bot must be added as admin with <code>Post Messages</code> permission. <code>getUpdates</code> delivers <code>channel_post</code> updates (not message) — Herald's current inbound handler set

Type	chat_id shape	Example	Notes
			covers private/group/supergroup; channel inbound is reserved for future work.

§6.2 Where chat_id flows through Herald

1. **At boot:** `HERALD_TGRAM_CHAT_ID` is the default outbound destination — `Send(ctx, msg)` posts there unless `msg.Recipient.ChannelUserID` overrides it.
2. **Per inbound event:** `Subscribe`'s handler reads `update.Message.Chat.ID` and stamps it into `InboundEvent.Recipient.ChannelUserID` (decimal string form). The `channelRouter` in `pherald/internal/inbound/dispatcher.go` keys reply destination by this stamped value — replies go back to the chat the inbound came from.
3. **Per outbound reply:** `SendReplyGeneric(ctx, recipient, body, replyToID, attachments)` parses `recipient.ChannelUserID` back to `int64` for the Bot API call.

§6.3 Permission requirements

Chat type	Bot must be	To do
private	the other end of the DM	read + send messages (always, no extra permission)
group	member	send messages (always); read all messages only when privacy mode is disabled
supergroup	member	send messages (always); read all messages only when privacy mode is disabled OR the bot is an admin with <code>can_read_all_group_messages</code>
channel	admin with <code>Post Messages</code>	send messages (read-only inbound is not implemented for channels in Herald's current path)

If the bot is kicked from a chat or its admin scope is revoked, the next Bot API call to that chat returns `403 Forbidden: bot was kicked from ...` — Herald surfaces this as a `subscriber-lost` signal (see §9.5).

§6.4 The -100 prefix is real

A common operator confusion: when reading the chat id from `getUpdates`, the JSON shows `"id": -1001234567890` — the `-100` IS part of the canonical id, NOT a formatting artifact. Strip nothing. Set `HERALD_TGRAM_CHAT_ID = '-1001234567890'` verbatim. Stripping the `-100` prefix gives you `1234567890` which Telegram resolves to a private chat with the corresponding user (if any), NOT your supergroup.

§6.5 Mapping to the spec

Spec V3 §43 catalogues the per-chat-type outbound routing matrix. Spec V3 §11.1 documents the Telegram-specific `Capabilities` (text + markdown + HTML + attachments + threads + interactive URL = true). Spec V3 §32.2 covers the inbound long-poll contract that drives the Wave 6 runtime.

§7. Reply threading + reply_to_message_id

Telegram supports two distinct threading mechanisms; Herald uses one of them, intentionally.

§7.1 reply_to_message_id — quoted reply (used)

The classic Telegram reply: tap a message in the client, choose "Reply", type the reply. Telegram renders the reply with a small quote-block of the original above. The Bot API field is `reply_to_message_id` — pass the original message's integer id, and `sendMessage` produces the same quoted-reply visual.

Herald's `tgram.SendReply` (in `send.go`) uses this field:

```
opts := &telebot.SendOptions{
    ParseMode: parseMode,
    ReplyTo:   &telebot.Message{ID: replyToID}, // →
              reply_to_message_id in the wire form
}
sent, err := a.bot.Send(chat, text, opts)
```

The wire-byte assertion (`commons_messaging/channels/tgram/send_reply_test.go`) pins the exact form-value name (`reply_to_message_id`) so a refactor that dropped the field would surface immediately. Wave 6 mutation gate (M3 in the paired §1.1 suite) drops the assignment to prove the detector catches it.

§7.2 message_thread_id — forum topics (supported, V1 stub)

Supergroups can be enabled as "forums" — each top-level message starts a new topic that has its own thread id. The Bot API field `message_thread_id` routes a new message into a specific topic. Herald supports this via `OutboundMessage.Thread.ThreadID` (decimal string), threaded through to `telebot.SendOptions.ThreadID`:

```
if msg.Thread != nil && msg.Thread.ThreadID != "" {
    if tid, terr := strconv.Atoi(msg.Thread.ThreadID); terr ==
        nil {
        opts.ThreadID = tid
    }
}
```

The V1 implementation does not auto-discover topic ids — the caller must provide one. Auto-discovery (reading the `message_thread_id` from inbound updates and threading it back into the matching `SendReply`) is reserved for a future HRD.

§7.3 Attachment fan-out (the M3 rationale)

Telegram supports multi-attachment "albums" via `sendMediaGroup` — up to 10 photos/videos in one wire call. Herald deliberately does NOT use albums. The Wave 6.5 mutation gate (M3) pins the rationale:

- **Reply threading:** an album appears in the chat as ONE bundle with one quoted-reply pointer. If you reply to it, the reply quotes the bundle, not any individual attachment. Herald's inbound dispatcher loses the per-attachment correlation under albums.

- **Mixed-MIME bundles:** `sendMediaGroup` requires every member of the bundle to be the same media type (all photos, all videos, all audio, OR all documents — never mixed). Herald's outbound messages can carry mixed-MIME attachments (a doc + a photo + a voice memo), so album-based fan-out would force MIME-bucketing AND multiple album calls — defeating the latency advantage.
- **Error granularity:** an album call that fails fails for ALL its attachments. The serialised-per-attachment path Herald uses gives per-attachment success/failure.

So `SendReplyGeneric` posts the text reply first (carrying `reply_to_message_id`), then sends each attachment as a separate `sendDocument` / `sendPhoto` / `sendVoice` reply at the same thread depth — each carrying its own `reply_to_message_id` pointing at the same original message. The visual in the Telegram client is a series of quoted replies forming a tidy "reply chain" under the original.

§7.4 The native vs generic `SendReply`

Two methods exist for historical reasons:

Method	Signature	When to use
<code>Adapter.SendReply</code>	<code>(ctx context.Context, chatID int64, body string, replyToID int, attachments []commons.Attachment) (int, error)</code>	Direct Telegram callers (typed ids).
<code>Adapter.SendReplyGeneric</code>	<code>(ctx context.Context, recipient commons.Recipient, body, replyToID string, attachments []commons.Attachment) (string, error)</code>	Channel-agnostic callers (<code>channels.Channel</code> interface).

The native method predates the Wave 7 channel interface; `SendReplyGeneric` is a thin adapter that parses the decimal string ids into typed forms and calls the native method. See `tgram.go:227..243`.

§8. Self-filter — preventing echo loops

Telegram bots are bidirectional in the same channel: they BOTH send and receive in the same chat. Without active filtering, every reply Herald posts would arrive back through `getUpdates` as a fresh subscriber message, be dispatched to Claude Code, generate another reply, be dispatched again — an exponential loop that saturates the 25-second long-poll within the first round-trip.

§8.1 The defence

`pherald listen` (specifically `tgram.Adapter.Subscribe` in `subscribe.go`) caches the bot's own `@username` ONCE at `Subscribe` boot:

```

bot, err := telebot.NewBot(telebot.Settings{
    Token:  a.botToken,
    Poller: &telebot.LongPoller{Timeout: 25 * time.Second},
})
// telebot.NewBot dispatches getMe synchronously, populating
    bot.Me.
selfUsername := ""
if bot.Me != nil {
    selfUsername = bot.Me.Username
}
if selfUsername == "" {
    return fmt.Errorf("tgram.Subscribe: bot.Me.Username unset
        after NewBot — getMe likely failed; refusing to boot
        without self-filter (echo-loop hazard)")
}
self := channels.SelfIdentity{Kind: channels.IdentityUsername,
    Value: selfUsername}

```

Then EVERY inbound message is checked against this cached identity via the channel-agnostic filter `channels.IsSelfEcho` (Wave 7 T4 — see `MESSENGER_CHANNELS.md` §6 for the cross-channel contract):

```

ev := buildInboundEvent(msg)
if stampAndIsSelfEcho(ev, msg, self) {
    return nil // drop — self-echo
}
return h.Handle(ev)

```

The mutation gate (Wave 6 T12 M1) pins this with a paired §1.1 test: removing the self-filter check causes the gate to mutate → assert-FAIL → restore. A live operator can never accidentally ship a self-filter-less binary.

§8.2 Fail-loud on empty identity

If `bot.Me.Username` is empty after `NewBot` (because `getMe` returned a degenerate user record, or because `telebot.Offline` was somehow enabled), `Subscribe` REFUSES to boot. There is NO `--allow-empty-identity` knob, NO `--skip-self-filter` knob, NO graceful-degradation path. A self-filter-less listener is a §107 PASS-bluff that LOOKS green but exponentially burns the operator's Claude quota on the first reply — the only safe behaviour is hard refusal.

§8.3 What if you legitimately need multi-bot conversation?

The filter is deliberately narrow: ONLY messages from THIS bot's own user record are dropped. A DIFFERENT bot in the same chat is kept and dispatched normally. So if you want bot-to-bot orchestration (a Slack bot pinging Herald's Telegram bot, etc.), this works out of the box — only the recursive self-loop is broken.

However, **bot-to-bot direct group messages are blocked at the Telegram API layer**, not the Herald layer. The memory file `telegram_bot_to_bot_wall` captures this: Telegram bots NEVER see another bot's group messages via `getUpdates`. A second-bot QA harness inside

Telegram is impossible at the Bot API tier — it would require an MTProto user-client (gotd/td-based). The QA workaround Herald uses is qaherald's dedicated user-account-driven scenario harness (see qaherald/cmd/qaherald/main.go).

§8.4 Where to look in the code

- commons_messaging/channels/tgram/subscribe.go:38..46 — the legacy shouldDropBotSelf helper (kept byte-for-byte because the Wave 6 M1 paired-mutation gate anchors its exact regex here).
 - commons_messaging/channels/tgram/subscribe.go:54..62 — stampAndIsSelfEcho — the live filter, used by every inbound handler.
 - commons_messaging/channels/selffilter.go — the channel-agnostic comparator IsSelfEcho.
 - commons_messaging/channels/tgram/subscribe_test.go — TestSubscribeBotSelfFilter pins the exact-text drop semantics.
-

§9. Troubleshooting cookbook

§9.1 401 Unauthorized

Symptom: every Bot API call returns
{ "ok": false, "error_code": 401, "description": "Unauthorized" }.

Root cause: token wrong, token revoked, or HERALD_TGRAM_BOT_TOKEN truncated by a quoting bug in .env.

Fix:

1. Confirm the env var is set in the running process — `cat /proc/$(pidof pherald)/environ | tr '\0' '\n' | grep HERALD_TGRAM` (Linux) or `ps eww -p $(pgrep pherald)` (macOS).
2. Re-fetch the token from BotFather: `/mybots` → <bot> → API Token.
3. If the token has been compromised, ROTATE via BotFather `/revoke`. There is no recovery — the old token is now invalid even if you reset the env var.
4. Restart `pherald listen`.

§9.2 400 Bad Request: chat not found

Symptom: `sendMessage` returns
{ "ok": false, "error_code": 400, "description": "Bad Request: chat not found" }.

Root cause: chat id is wrong-format (e.g. 1234567890 for a supergroup that needs -1001234567890), or the bot has not been added to the target chat.

Fix:

1. Verify the chat id format matches §6.1.
2. Confirm the bot is a member of the chat: `curl -s "https://api.telegram.org/bot$TOKEN/getChat?chat_id=$ID" | jq` — if it returns `Bad Request: chat not found`, the bot is not in the chat (or `chat_id` is wrong).

3. For groups/supergroups/channels: add the bot via §2.3.

§9.3 429 Too Many Requests

Symptom: Bot API call returns

```
{"ok":false,"error_code":429,"description":"Too Many Requests:
retry after N","parameters":{"retry_after":N}}.
```

Root cause: Telegram rate-limiter triggered. Bot API has both global (getUpdates-side) and per-chat (sendMessage-side) rate caps.

Fix:

1. The `retry_after` field is the canonical signal — sleep for at least that many seconds before retrying.
2. Herald's adapter does NOT auto-backoff on 429 (yet — reserved HRD); a caller burst that hits 429 will surface the error. A future HRD will add exponential backoff with `retry_after` honour.
3. If you see persistent 429s, reduce send concurrency or add intentional spacing between sends to the same chat.

§9.4 413 Request Entity Too Large / 400 file is too big

Symptom: `sendDocument` returns 413 (or 400 file is too big for inbound `getFile`).

Root cause: exceeded the 50 MB upload cap (or 20 MB download cap) — see §5.

Fix:

1. Split the file (split + transmit as multiple messages).
2. Compress (zstd, xz, gzip) before upload.
3. For documents > 50 MB: run a local Bot API server (§5.4) — reserved HRD.

§9.5 403 Forbidden: bot was kicked from the group chat

Symptom: outbound to a previously-working chat returns

```
{"ok":false,"error_code":403,"description":"Forbidden: bot was
kicked from the group chat"}.
```

Root cause: a chat admin removed the bot, or the chat was deleted, or the user (in a DM) blocked the bot.

Fix:

1. This is the canonical "subscriber lost" signal — Herald should mark the subscriber inactive in the registry and stop attempting delivery.
2. To recover: re-add the bot via §2.3.
3. There is no automatic re-subscribe — by design, an explicit operator action is required to mark the subscriber active again.

§9.6 Echo loop — bot replying to itself

Symptom: the bot replies once, then a second, third, fourth reply arrives within seconds; the long-poll never quiesces.

Root cause: `bot.Me.Username` was empty at Subscribe boot OR the self-filter check was bypassed by a regression.

Fix:

1. Confirm Subscribe REFUSED to boot if `bot.Me.Username` was empty — if it booted with empty username, file a SECURITY bug (the §107 fail-loud invariant was broken).
2. Verify the live username: `curl -s "https://api.telegram.org/bot$TOKEN/getMe" | jq '.result.username'` — must be non-empty.
3. Grep your pherald binary's logs for the `bot.Me.Username= boot` line — confirm it carries your bot's canonical username (without @).
4. If you can reproduce in dev, run with race detector + log every dispatch — the loop will be visible within a few seconds.

§9.7 Long-poll stuck — no updates for >25s

Symptom: `pherald listen` is running but no inbound messages flow even when you send them in the chat.

Root causes (in descending order of likelihood):

1. **Privacy mode is ON in a group** (§2.2). Disable via `BotFather /setprivacy`.
2. **The bot is not in the chat you think it is** (§9.2 — silent variant; `getUpdates` returns empty rather than erroring).
3. **Network firewall to api.telegram.org:443**. Test: `curl -v https://api.telegram.org/bot$TOKEN/getMe`. If it hangs, your network blocks Telegram (corporate proxies are the usual suspect).
4. **A webhook is configured** — `getUpdates` returns updates only if NO webhook is set. Delete via `curl -X POST https://api.telegram.org/bot$TOKEN/deleteWebhook`. See §9.10 below.

§9.8 409 Conflict: terminated by other getUpdates request

Symptom: `pherald listen` boots, then aborts with `Conflict: terminated by other getUpdates request`; make sure that only one bot instance is running.

Root cause: Telegram allows ONLY ONE `getUpdates` consumer per token at a time. A second `pherald listen` instance, a forgotten dev shell, a CI job running the integration test, or a third-party debug client polling with your token will trigger this.

Fix:

1. Find the competing consumer: `pgrep -fa 'pherald listen'`.
2. Kill it: `pkill -f 'pherald listen'`.
3. Wait a few seconds (Telegram closes the conflicting session) and restart your `pherald listen`.
4. If you also have a webhook configured for the same token, `getUpdates` will conflict with it — `deleteWebhook` first (§9.10).

§9.9 Privacy mode — bot only sees commands in groups

Symptom: DMs work fine; group messages never arrive at the handler EXCEPT commands like `/foo` or messages that mention `@your_bot`.

Root cause: privacy mode is ON (the default for new bots).

Fix: DM `@BotFather`, `/setprivacy`, select your bot, `Disable`. Restart `pherald listen`. The bot will now see every message in groups it is a member of.

§9.10 Webhook conflicts with long-poll

Symptom: `getUpdates` returns the same updates repeatedly OR returns empty + 409 Conflict arrives without a competing process.

Root cause: a webhook was previously configured on this token. Bot API does NOT permit `getUpdates` while a webhook is active.

Fix:

```
curl -s "https://api.telegram.org/bot${HERALD_TGRAM_BOT_TOKEN}/getWebhookInfo" | jq
```

If the `url` field is non-empty, delete the webhook:

```
curl -X POST "https://api.telegram.org/bot${HERALD_TGRAM_BOT_TOKEN}/deleteWebhook"
```

Confirm with another `getWebhookInfo` — `url` must be empty. Then restart `pherald listen`.

§9.11 Zero-byte / .part residue in the inbox

Symptom: `~/ .herald/inbox/tgram/dl-*.part` files remain after a download, or content-addressed final files are zero-byte.

Root cause: an inbound attachment stream was interrupted before `os.Rename` completed (network drop, process kill, disk full).

Fix:

1. Delete the `.part` files — they are deliberate temp residue; the next download starts fresh.
2. Zero-byte FINAL files would indicate a regression in `DownloadAttachment` — file a bug (the `attachments_test.go` invariant should have caught it pre-merge).

§9.12 Attachment ends up in slack/ subdir (wrong inbox)

Symptom: a Telegram-delivered photo lands at `~/ .herald/inbox/slack/<sha>.jpg` instead of `~/ .herald/inbox/tgram/<sha>.jpg`.

Root cause: a refactor caused `Adapter.Name()` to drift from the registered channel name OR `HERALD_INBOX_DIR` is set to a path that overlaps another deployment.

Fix: see `MESSENGER_CHANNELS.md` §9.5.

§10. Pre-deploy operator audit

Run through this checklist before every fresh deploy or after rotating credentials.

§10.1 Environment

☐

HERALD_TGRAM_BOT_TOKEN is exported in the running pherald's environment. Format matches `<numeric-id>:<alphanumeric-hmac>`.

☐

HERALD_TGRAM_CHAT_ID is exported. Format matches §6.1 for the chat type (positive for DM, negative for group, `-100...` for supergroup/channel).

☐

HERALD_INBOX_DIR is either unset (default `~/ .herald/inbox`) or points at a writable directory with at least worst-case-burst disk headroom ($50\text{ MB} \times \text{expected concurrent uploads}$).

☐

No tokens appear in `git ls-files | xargs grep -l '<your-token-prefix>'` (manual spot-check).

§10.2 Bot configuration

☐

`/setprivacy` is `Disable` IF the bot is in groups and is meant to see all messages.

☐

Bot is a member of every target chat (`curl -s "https://api.telegram.org/bot$TOKEN/getChat?chat_id=$ID" | jq` returns `ok: true`).

☐

No competing `getUpdates` consumer is running (`pgrep -fa 'pherald listen'` returns at most one match).

☐

No webhook is configured (`curl -s "$/getWebhookInfo" | jq '.result.url'` returns `""`).

§10.3 Registry resolution

☐

HERALD_CHANNELS either is unset (defaults to `tgram`) or contains `tgram` as one of the comma-separated values.

☐

`grep 'channels.Register' commons_messaging/channels/*/*.go` lists `tgram` (sanity-check the blank-import is intact).

§10.4 Filesystem

☐

`~/ .herald/inbox/` exists with mode `0700`.

☐

~/ .herald/inbox/tgram/ either exists with 0700 or will be auto-created on first attachment.



Disk has worst-case-burst headroom (50 MB × concurrent uploads × 2 for safety).

§10.5 §107.x evidence



Plan the run-id directory: docs/qa/HRD-<NNN>-<TS>Z/ (timestamp UTC).



For pherald listen smoke runs, pass --qa-out-dir docs/qa/HRD-NNN-<TS>Z/ so the journal lands in the artefact directory.



For manual operator-driven runs, screenshot both directions of the chat (subscriber message + bot reply) and commit the screenshots in the run-id directory.



For qaherald-driven QA runs (Wave 5+), the transcript + attachments tree auto-lands under docs/qa/qaherald-<TS>/ — confirm presence before tagging.

§10.6 First-message smoke



Build the binary: go build -o /tmp/pherald ./pherald/cmd/pherald.



Boot: /tmp/pherald listen --qa-out-dir docs/qa/HRD-NNN-<TS>Z/2>&1 | tee /tmp/pherald-listen.log.



Boot line pherald listen: bot.Me.Username=... shows non-empty username (self-filter active).



Type a one-word test message in the target chat (e.g. hello). Within ~25s, a reply arrives in the chat.



The reply quotes the original (reply_to_message_id set).



docs/herald/diary/main.md contains the new conversation entries (in + out).



docs/qa/HRD-NNN-<TS>Z/journal.jsonl contains one line per dispatch.



Ctrl-C gracefully shuts down (no zombie process).

§11. References

§11.1 Spec V3

- §11.1 — Telegram channel Capabilities declaration (text + markdown + HTML + attachments + threads + interactive URL = true; AttachmentMaxMiB = 50; DeliveryCeiling = DeliveryRouted).

- **§32.2** — inbound long-poll contract; `getUpdates` timeout = 25s; observational 30s safety-net timer.
- **§32.9** — anti-echo-loop self-filter mandate (the §107 binding for `bot.Me.Username` capture + `IsSelfEcho` fan-out).
- **§43** — channel + flavor catalogue; per-chat-type routing matrix.
- **§107.x** — `docs/qa/<run-id>/` evidence mandate (Helix §11.4.83 cascade).

§11.2 HRDs

HRD	Wave	Status	Description
HRD-011	1	LIVE	Telegram outbound substrate — <code>Send</code> , <code>MarkdownV2</code> , attachments, persistence.
HRD-100	6	<code>in_progress</code> (awaiting T10b live evidence)	<code>pherald listen</code> inbound runtime — <code>getUpdates</code> long-poll, Claude Code dispatch, <<<HERALD-REPLY>>> action router.
HRD-101	6	LIVE	Wave 6 paired §1.1 mutation gate covering inbound runtime (M1=self-filter, M2=envelope, M3=attachment fan-out).
HRD-110	7	LIVE	Extract <code>channels.Channel</code> interface (richer inbound contract embedding §11.0).
HRD-111	7	LIVE	<code>init()</code> -based channel registry.
HRD-112	7	LIVE	Per-channel inbox subdirs (<code>~/herald/inbox/tgram/</code>).
HRD-113	7	LIVE	Generalize bot self-filter via <code>BotSelfIdentity</code> + <code>IsSelfEcho</code> .
HRD-114	7	LIVE	Multi-channel <code>pherald listen</code> fan-in.
HRD-133	<code>tgram-completeness</code>	open	Telegram capabilities matrix completeness review.
HRD-134	<code>tgram-completeness</code>	open	Rationale for not auto-splitting 4096+ char messages.
HRD-135	<code>tgram-completeness</code>	open	Local Bot API server support (lifts 50 MB cap).
HRD-136	<code>tgram-completeness</code>	open	Forum-topic auto-threading (inbound <code>message_thread_id</code> → outbound <code>SendReply</code>).
HRD-137	<code>tgram-completeness</code>	open	Channel-post inbound handler (<code>update.channel_post</code>).
HRD-138	<code>tgram-completeness</code>	open	This guide.

§11.3 Source files

- `commons_messaging/channels/tgram/tgram.go` — Adapter, `New / NewWithCreds / NewWithStorage`, `init()` registry binding, `BotSelfIdentity / SendReplyGeneric / DownloadAttachment` (channel interface methods).
- `commons_messaging/channels/tgram/send.go` — `Send` (outbound), native `SendReply(chatID int64, replyToID int)`.

- `commons_messaging/channels/tgram/subscribe.go` — Subscribe long-poll loop, `stampAndIsSelfEcho` filter, `OnText/OnPhoto/OnDocument/OnVoice` handlers.
- `commons_messaging/channels/tgram/attachments.go` — `DownloadAttachment(ctx, bot, fileID, mime)`.
- `commons_messaging/channels/tgram/healthcheck.go` — `HealthCheck` (calls `getMe`).
- `commons_messaging/channels/tgram/persist.go` — `outbound_delivery_evidence` persistence.
- `commons_messaging/channels/channel.go` — the eight-method channel interface.
- `commons_messaging/channels/selffilter.go` — `StampSender`, `IsSelfEcho` (channel-agnostic).
- `commons_messaging/channels/inbox.go` — `WriteContentAddressed`, `InboxDir`, `MimeToExt`.
- `pherald/cmd/pherald/listen.go` — `loadEnabledChannels`, `perChannelConfig`, `runListen`, `channelRouter`.
- `pherald/internal/inbound/dispatcher.go` — Claude Code dispatch + reply-action router.

§11.4 Plan + design docs

- `docs/superpowers/plans/2026-05-21-wave6-cc-headless-bridge.md` — Wave 6 inbound runtime plan.
- `docs/superpowers/plans/2026-05-27-wave7-generic-messenger.md` — Wave 7 multi-channel framework.
- `docs/CONTINUATION.md` §3 — live-test handoff prompt (operator-supplied credentials).

§11.5 Sibling guides

- [MESSENGER_CHANNELS.md](#) — multi-channel framework, registry semantics, multi-channel `pherald listen` semantics, `HERALD_CHANNELS` env var.
- [OPERATOR_CREDENTIALS.md](#) — umbrella credentials guide; the `.env` resolution order; the audit checklist.
- [messengers/TELEGRAM.md](#) — the original HRD-011 setup walkthrough (Step 1..Step 7) — more verbose for first-time operators; this guide is the operator-deep-dive complement.
- [messengers/SLACK.md](#) — Slack-specific deep-dive (HRD-115).
- [HERALD_CONSTITUTION.md](#) — §107 end-user-usability covenant + §107.x evidence mandate.
- [dispatchers/CLAUDE_CODE.md](#) — Claude Code dispatcher (HRD-012) — the other half of the inbound round-trip.

§11.6 Constitutional anchors

- Helix Universal Constitution §11.0 — outbound `Channel` contract.
- Helix Universal Constitution §11.4 / §11.4.4 / §11.4.5 / §11.4.6 — anti-bluff captured-evidence mandate.
- Helix Universal Constitution §11.4.83 — `docs/qa/<run-id>/evidence cascade` (Herald §107.x).
- Helix Universal Constitution §11.4.85 — stress + chaos test mandate.
- Herald §107 — end-user-usability covenant.

- Herald §107.x — docs/qa evidence mandate.
- Herald §107.y — working-tree quiescence rule.

§11.7 External

- [Telegram Bot API documentation](#) — canonical reference for every endpoint Herald calls (getMe, getUpdates, getChat, getFile, sendMessage, sendPhoto, sendDocument, sendVoice, deleteWebhook, getWebhookInfo).
- [BotFather](#) — bot creation + management.
- [@RawDataBot](#) — third-party chat-id helper bot (use with caution; vet before sharing sensitive content).
- [Telegram Bot API server \(open source\)](#) — the local server that lifts the 50 MB cap (HRD-135 reserved).

End of guide.